AEM341 Project Part2

Collin Atkins

Department of Aerospace Engineering and Mechanics at the University of Alabama, Tuscaloosa, AL, 35487, USA

I. Objective

The objective of this project was to compare theoretical bidirectional bending results for an unsymmetrical L-shaped cantilever beam with the experimental strain and displacement data from the AEM351 bidirectional bending lab. This project aimed to do the following as well: quantify the mismatch between theory and experiment using the Average Percent Relative Error (APRE), determine an improved Young's modulus under the assumption of perfect clamping, and identify a set of non-perfect clamping boundary conditions that better match the measured beam deflections. Evaluating a combined fitting approach for both material property and boundary-condition inference was an additional objective.

II. Theory and Method

The L-shaped cantilever beam's bidirectional bending response was analyzed using Bernoulli-Euler beam theory for an unsymmetrical cross-section. Bending in the beam produces coupled displacement in both the vertical and horizontal directions because its section has a nonzero product of inertia. Beam deflections and bending stress at the strain-gage locations were computed using centroid location, area moments of inertia, and product of inertia. Experimental strain and displacement measurements from the AEM351 bidirectional bending lab were then compared with the theoretical bidirectional bending results.

The beam was modeled for the initial comparison using the nominal aluminum Young's modulus and a perfect clamping condition at the fixed end of the beam. MATLAB was used to plot the theoretical vertical displacement, horizontal displacement, stress at the side gages, stress at the base gages, strain at the side gages, and strain at the base gages against the experimental data from the AEM351 bidirectional bending lab. The discrepancy between the theoretical and experimental results was quantified for these six groups using APRE.

The agreement between the experimental results and the theoretical results was then improved using an inverse-fitting approach. First, a revised Young's modulus was determined by minimizing the strain and stress discrepancy. This was done while still assuming perfect clamping. Next, non-perfect clamping boundary conditions were introduced. This was done by allowing a nonzero root displacement and a nonzero root slope for both displacement directions. These boundary-condition constants that were used were selected to minimize the deflection mismatch between theory and experiment. For the extra problem, a combined fitting measure was also used to determine both an effective Young's modulus and a set of boundary conditions that improved the agreement between experiment and theory for both strain and deflection.

III. MATLAB Program

The MATLAB program was used to produce theoretical results and to compare these results to the corresponding experimental results from the AEM351 bidirectional bending lab. The beam geometry, material properties, strain-gage locations, and experimental measurements for the selected load case were defined in the program. It then calculated the theoretical vertical displacement, horizontal displacement, stress, and strain distributions along the L-shaped

cantilever beam. The program also generated all of the required comparison plots and calculated the APRE for each of the six experimental and theoretical data groups.

Additionally, the MATLAB program was used to perform the inverse fitting steps. Under the assumption of perfect clamping, a fitted Young's modulus was determined. A set of non-perfect clamping boundary conditions was also determined by fitting the measured deflection data. A combined fitting approach was evaluated as well for the extra problem. This adjusted both the modulus and the boundary conditions to improve agreement between theory and experiment.

IV. Input Data

The input data for this part of the project were based on the same L-shaped cantilever beam geometry from Project Part 1 and the experimental strain and displacement data from the AEM351 bidirectional bending lab. A beam length of 1 m, an aluminum elastic modulus of 68.9 GPa as the nominal value, and a 12 lb load case were used for the analysis in the MATLAB program. The beam was evaluated at the ten specified strain-gage locations in Table 2 and at the five required displacement sample locations along the span. The specific cantilever beam geometry, strain-gage locations, and L-shaped cross-section dimensions are shown in Fig. 1.

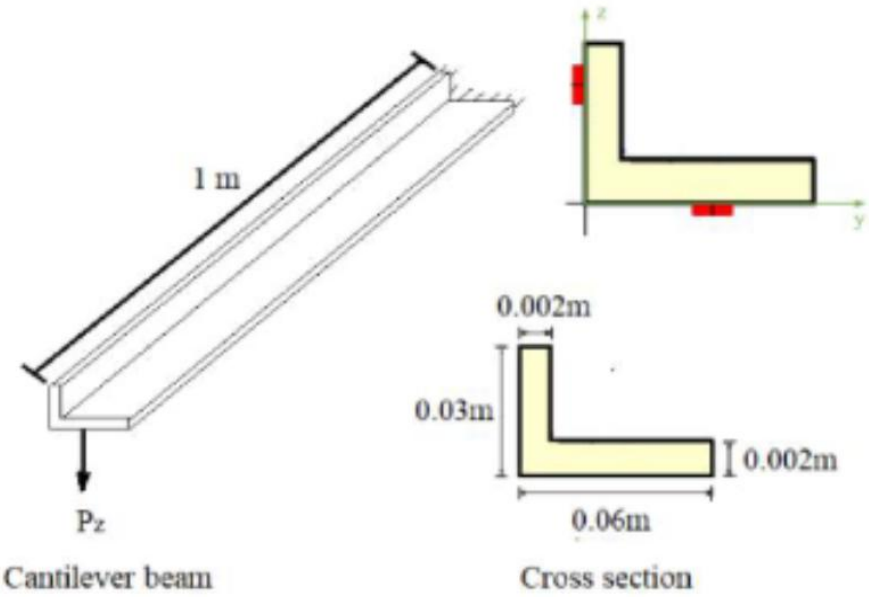


Fig. 1. Specific Cantilever Beam Geometry and Cross-Section for Project Part 2

The experimental bidirectional bending lab dataset provided strain measurements for ten gages. It also provided horizontal and vertical displacement measurements for five x-locations. 6 lb and 12 lb were the available load levels. The 12-lb case was selected for the presented results. All the strain data were given in microstrain, and so, no strain-gage factor correction was required. The beam and material input data are listed in Table 1. The strain-gage locations are listed in Table 2. The experimental strain and displacement data for the selected load case are listed in Table 3.

Table 1. Beam and Material Input Data

Parameter	Value
Vertical leg height, h_1	0.030 m
Horizontal leg height, h_2	0.060 m
Vertical leg thickness, t_1	0.002 m
Horizontal leg thickness, t_2	0.002 m
Beam length, L	1.000 m
Nominal elastic modulus, E	68.9 GPa
Selected load case	12 lb
Load direction	-z direction
Span discretization	1 mm

Table 2. Strain-Gage Locations

Strain Gage	x (m)	y (m)	z (m)
1	0.0348	0.0000	0.0150
2	0.2761	0.0000	0.0150
3	0.5047	0.0000	0.0150
4	0.7333	0.0000	0.0150
5	0.9746	0.0000	0.0150
6	0.0348	0.0200	0.0000
7	0.2761	0.0200	0.0000
8	0.5047	0.0200	0.0000
9	0.7333	0.0200	0.0000
10	0.9746	0.0200	0.0000

Table 3. Experimental Strain and Displacement Data for the Selected 12-lb Load Case

x (m)	Side-Gage Strain, ϵ ($\mu\epsilon$)	Base-Gage Strain, ϵ ($\mu\epsilon$)	Vertical Displacement, w (mm)	Horizontal Displacement, v (mm)
0.0348	309	-630	-0.933	-0.069
0.2761	136	-326	-10.048	-0.730
0.5047	99	-239	-20.136	-3.257
0.7333	69	-119	-31.087	-5.306
0.9746	32	-58	-44.222	-8.324

V. Results

The theoretical results that were produced by that MATLAB program were compared with the experimental strain and displacement measurements from the lab for the selected 12-lb case. The nominal aluminum modulus and a perfect clamping condition were used for the initial comparison. The six required comparison groups were the following: vertical displacement, horizontal displacement, stress at the side gages, stress at the base gages, strain at the side gages, and strain at the base gages. APRE values were calculated for each of these six groups. The baseline comparison between theoretical and experimental results for the selected 12-lb load case is shown in Fig. 2. The APRE values for the six baseline comparison groups are listed in Table 4.

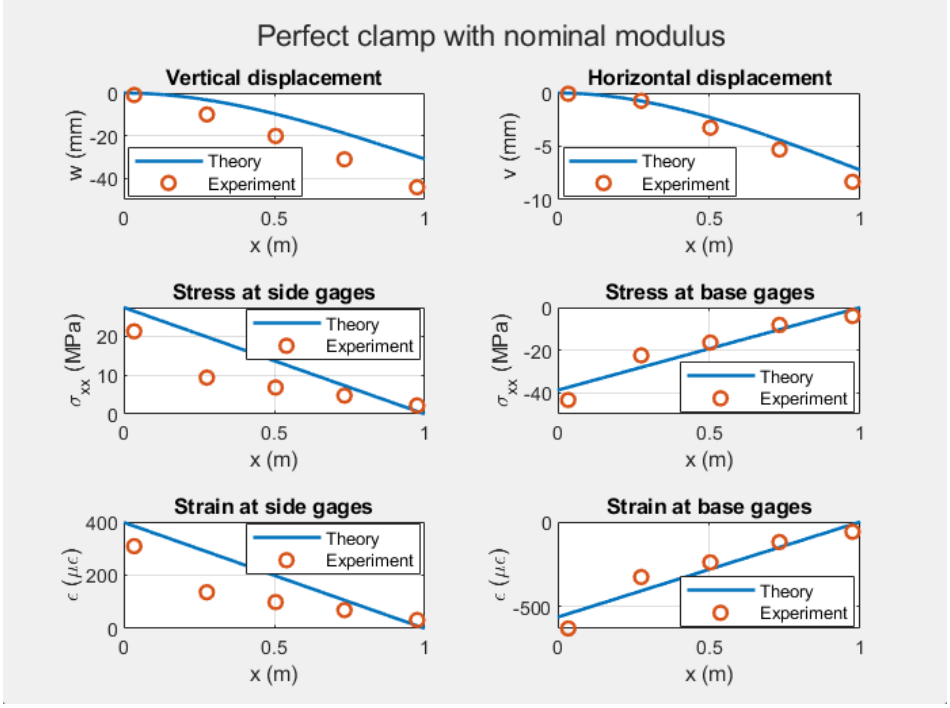


Fig. 2. Baseline Comparison between Theoretical and Experimental Results for the Selected 12-lb Load Case

Table 4. APRE Values for the Six Baseline Comparison Groups

Case	Vertical Displacement APRE (%)	Horizontal Displacement APRE (%)	Stress at Side Gages APRE (%)	Stress at Base Gages APRE (%)	Strain at Side Gages APRE (%)	Strain at Base Gages APRE (%)
Baseline (perfect clamp with nominal modulus)	72.151	22.706	-34.578	-1.5952	-34.578	-1.5952

A revised Young's modulus was then determined by fitting the strain and stress response. This was done while maintaining the perfect clamping assumption. Compared to the nominal modulus, the fitted modulus was greater, meaning that the original material-property assumption did not provide the best agreement with the experimental data. The fitted Young's modulus result for the perfect-clamp case is listed in Table 5. The comparison between theoretical and experimental results using the fitted Young's modulus and perfect clamping is shown in Fig. 3.

Table 5. Fitted Young's Modulus Result for the Perfect-Clamp Case

Quantity	Value
Nominal Young's modulus, E	6.890000×10^{10} Pa
Fitted Young's modulus, E	7.262615×10^{10} Pa

Percent increase from nominal	5.41%

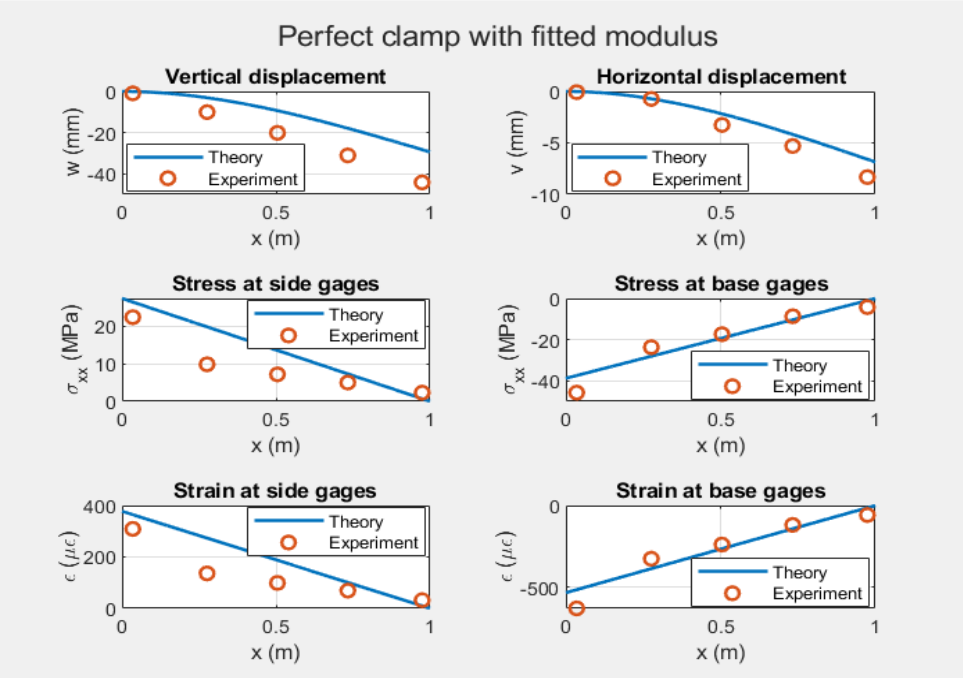


Fig. 3. Comparison between Theoretical and Experimental Results using the Fitted Young's Modulus and Perfect Clamping

Next, non-perfect clamping boundary conditions were introduced to account for possible clamp compliance. This was done by allowing a nonzero root displacement and a nonzero root slope in both displacement directions. The agreement between the theoretical and experimental deflection data was significantly improved by these fitted boundary conditions. The stress and strain trends remained governed by the beam bending model. The fitted non-perfect clamping boundary conditions are listed in Table 6. The comparison between theoretical and experimental results using fitted non-perfect clamping boundary conditions is shown in Fig. 4.

Table 6. Fitted Non-Perfect Clamping Boundary Conditions

Boundary Condition	Value
dw/dx(0)	-1.389185×10^{-2} rad
w(0)	-1.909706×10^{-3} m
dv/dx(0)	-1.519913×10^{-3} rad
v(0)	1.125730×10^{-4} m

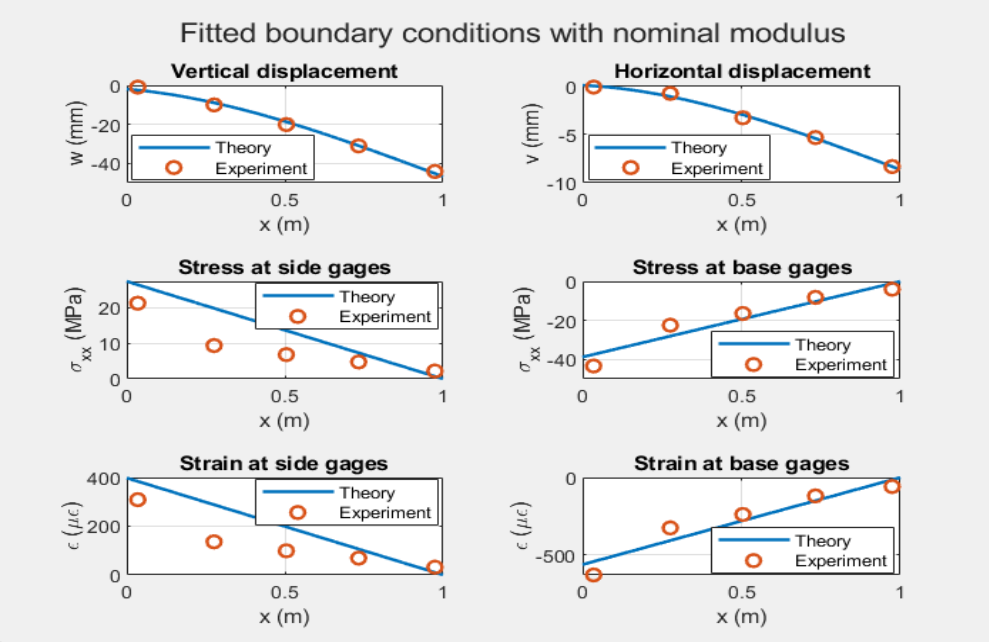


Fig. 4. Comparison between Theoretical and Experimental Results using the Fitted Non-Perfect Clamping Boundary Conditions

For the extra problem, an effective modulus and a set of boundary conditions were determined by applying a combined fitting approach too. This improved agreement in strain and deflection simultaneously. The best overall compromise among the model cases considered was found with the combined-fit case. The RMSE summary for the baseline, fitted-modulus, fitted-boundary, and combined-fit cases is listed in Table 7. The combined-fit modulus, boundary conditions, and fit measure are listed in Table 8. The comparison between theoretical and experimental results using the combined fitted modulus and boundary conditions is shown in Fig. 5.

Table 7. RMSE Summary for the Baseline, Fitted-Modulus, Fitted-Boundary, and Combined-Fit Cases

Case	Vertical Displacement RMSE (mm)	Horizontal Displacement RMSE (mm)	Stress at Side Gages RMSE (MPa)	Stress at Base Gages RMSE (MPa)	Strain at Side Gages RMSE (με)	Strain at Base Gages RMSE (με)
Baseline	10.097	0.8518	6.1876	4.2044	89.806	61.021
Fitted modulus	10.897	1.0425	5.7396	4.5887	79.03	63.182
Fitted boundary	1.1339	0.21173	6.1876	4.2044	89.806	61.021
Combined fit	0.88417	0.22563	5.1448	5.6274	65.93	72.115

Table 8. Combined-Fit modulus, Boundary Conditions, and Fit Measure

Quantity	Value

Combined-fit Young's modulus, E	7.803362 x 10^{10} Pa
dw/dx(0)	-1.765888 x 10^{-2} rad
w(0)	-1.455116 x 10^{-3} m
dv/dx(0)	-2.398532 x 10^{-3} rad
v(0)	2.188236 x 10^{-4} m
Combined fit measure	1.316184 x 10^{-2}

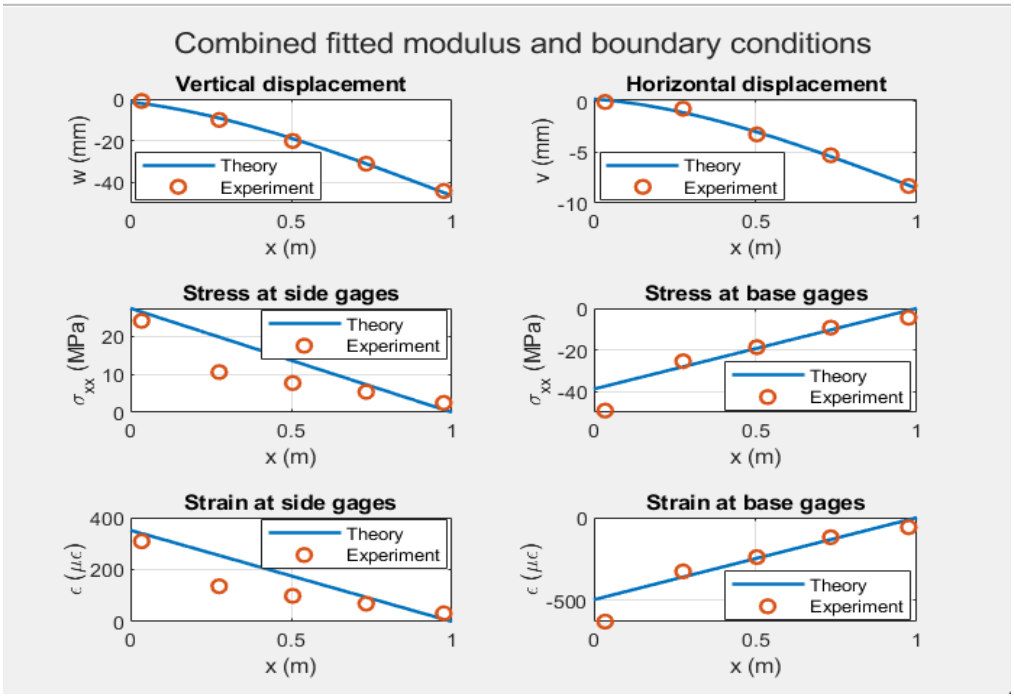


Fig. 5. Comparison between Theoretical and Experimental Results using the Combined Fitted Modulus and Boundary Conditions

VI. Discussion

The baseline perfect-clamp model with the nominal aluminum modulus captured the overall trends of the experimental response; however, noticeable differences remained between the theoretical results and the measured results. The vertical deflection in the displacement results had the largest disagreement. It suggested that the ideal fixed-end assumption did not fully represent the actual experimental boundary condition. On the other hand, the stress and strain trends were captured more reasonably, even so, differences were still present between theory and experiment. All of these comparisons satisfy this project's objective: to improve the material-property and boundary-condition assumptions used in the beam model.

The strain and stress improved when only Young's modulus was fitted under the perfect-clamp assumption. However, the displacement mismatch was not significantly reduced. On the other hand, allowing non-perfect clamping boundary conditions improved the agreement between the theoretical and experimental deflection results by a lot. This means that clamp compliance and root rotation were important sources of error in the baseline model. The combined fit for the extra problem gave the best overall compromise between experimental and theoretical displacement agreement and experimental and theoretical strain agreement. APRE

was reported, and RMSE was used as a supplemental measure to help interpret the fit quality more clearly.

VII. Conclusion

This project successfully compared the theoretical bidirectional-bending response of the L-shaped cantilever beam with the experimental strain and displacement data for the selected load case from the AEM351 bidirectional bending lab. For the six comparison groups, APRE values were determined. A set of non-perfect clamping boundary conditions was identified as well. These boundary conditions gave a much better match between the theoretical and experimental deflection results. For the extra problem, the best overall agreement between theory and experiment was provided by the combined-fit model.

AEM341 Project Part2

Collin Atkins 4/24/2026

```
function results = AEM341_Project2_Main(inputs)

if nargin == 0
    inputs = build_default_inputs();
end

validate_inputs(inputs);

section = get_l_section_properties(inputs.section);

validate_stress_points(inputs.analysis.stress_points, section);

load_case = get_selected_load_case(inputs.loads, inputs.experimental);

baseline = get_case_results( ...
    section, ...
    inputs.beam, ...
    load_case, ...
    inputs.analysis, ...
    zeros(4, 1));

baseline_experimental = get_experimental_results( ...
    load_case, ...
    inputs.analysis, ...
    inputs.beam.elastic_modulus, ...
    inputs.experimental);

baseline_apre = get_apre_summary( ...
    baseline, ...
    baseline_experimental);

best_modulus = fit_elastic_modulus( ...
    baseline.theory.stress_all_pa, ...
    baseline_experimental.strain_all_micro);

modulus_beam = inputs.beam;

modulus_beam.elastic_modulus = best_modulus;

modulus_case = get_case_results( ...
    section, ...
    modulus_beam, ...
    load_case, ...
    inputs.analysis, ...
    zeros(4, 1));

modulus_experimental = get_experimental_results( ...
    load_case, ...
```

```
    inputs.analysis, ...
    best_modulus, ...
    inputs.experimental);

modulus_apre = get_apre_summary( ...
    modulus_case, ...
    modulus_experimental);

boundary_constants = fit_boundary_conditions( ...
    baseline.theory.sample_positions, ...
    baseline.theory.sample_vertical_disp_m, ...
    baseline.theory.sample_horizontal_disp_m, ...
    baseline_experimental.vertical_disp_m, ...
    baseline_experimental.horizontal_disp_m);

boundary_case = get_case_results( ...
    section, ...
    inputs.beam, ...
    load_case, ...
    inputs.analysis, ...
    boundary_constants);

boundary_apre = get_apre_summary( ...
    boundary_case, ...
    baseline_experimental);

combined_fit = fit_combined_model( ...
    section, ...
    inputs.beam, ...
    load_case, ...
    inputs.analysis, ...
    baseline_experimental);

combined_beam = inputs.beam;

combined_beam.elastic_modulus = combined_fit.elastic_modulus;

combined_case = get_case_results( ...
    section, ...
    combined_beam, ...
    load_case, ...
    inputs.analysis, ...
    combined_fit.boundary_constants);

combined_experimental = get_experimental_results( ...
    load_case, ...
    inputs.analysis, ...
    combined_fit.elastic_modulus, ...
    inputs.experimental);

combined_apre = get_apre_summary( ...
    combined_case, ...
    combined_experimental);
```

```
baseline_rmse = get_rmse_summary( ...
    baseline, ...
    baseline_experimental);

modulus_rmse = get_rmse_summary( ...
    modulus_case, ...
    modulus_experimental);

boundary_rmse = get_rmse_summary( ...
    boundary_case, ...
    baseline_experimental);

combined_rmse = get_rmse_summary( ...
    combined_case, ...
    combined_experimental);

results = struct();

results.inputs = inputs;
results.section = section;
results.load_case = load_case;
results.baseline = baseline;
results.baseline_experimental = baseline_experimental;
results.baseline_apre = baseline_apre;
results.best_modulus_pa = best_modulus;
results.modulus_case = modulus_case;
results.modulus_experimental = modulus_experimental;
results.modulus_apre = modulus_apre;
results.boundary_constants = boundary_constants;
results.boundary_case = boundary_case;
results.boundary_apre = boundary_apre;
results.combined_fit = combined_fit;
results.combined_case = combined_case;
results.combined_experimental = combined_experimental;
results.combined_apre = combined_apre;
results.baseline_rmse = baseline_rmse;
results.modulus_rmse = modulus_rmse;
results.boundary_rmse = boundary_rmse;
results.combined_rmse = combined_rmse;
results.apre_table = build_apre_table( ...
    baseline_apre, ...
    modulus_apre, ...
    boundary_apre, ...
    combined_apre);
rmse_table = table( ...
    [baseline_rmse.vertical_disp_mm; ...
     modulus_rmse.vertical_disp_mm; ...
     boundary_rmse.vertical_disp_mm; ...
     combined_rmse.vertical_disp_mm], ...
    [baseline_rmse.horizontal_disp_mm; ...
     modulus_rmse.horizontal_disp_mm; ...
     boundary_rmse.horizontal_disp_mm; ...
     combined_rmse.horizontal_disp_mm], ...
    [baseline_rmse.stress_side_mpa; ...
```

```
     modulus_rmse.stress_side_mpa; ...
     boundary_rmse.stress_side_mpa; ...
     combined_rmse.stress_side_mpa], ...
    [baseline_rmse.stress_base_mpa; ...
     modulus_rmse.stress_base_mpa; ...
     boundary_rmse.stress_base_mpa; ...
     combined_rmse.stress_base_mpa], ...
    [baseline_rmse.strain_side_micro; ...
     modulus_rmse.strain_side_micro; ...
     boundary_rmse.strain_side_micro; ...
     combined_rmse.strain_side_micro], ...
    [baseline_rmse.strain_base_micro; ...
     modulus_rmse.strain_base_micro; ...
     boundary_rmse.strain_base_micro; ...
     combined_rmse.strain_base_micro], ...
    'VariableNames', { ...
     'vertical_disp_mm', ...
     'horizontal_disp_mm', ...
     'stress_side_mpa', ...
     'stress_base_mpa', ...
     'strain_side_micro', ...
     'strain_base_micro'}, ...
    'RowNames', { ...
     'baseline', ...
     'fitted_modulus', ...
     'fitted_boundary', ...
     'combined_fit'});
results.rmse_table = rmse_table;
results.stress_table = build_stress_comparison_table( ...
    baseline, ...
    baseline_experimental, ...
    modulus_case, ...
    modulus_experimental);
results.deflection_table = build_deflection_comparison_table( ...
    baseline, ...
    baseline_experimental, ...
    boundary_case, ...
    combined_case);

display_summary(results);

plot_comparison_set( ...
    baseline, ...
    baseline_experimental, ...
    'Perfect clamp with nominal modulus');

plot_comparison_set( ...
    modulus_case, ...
    modulus_experimental, ...
    'Perfect clamp with fitted modulus');

plot_comparison_set( ...
    boundary_case, ...
    baseline_experimental, ...
```

```
    'Fitted boundary conditions with nominal modulus');

plot_comparison_set( ...
    combined_case, ...
    combined_experimental, ...
    'Combined fitted modulus and boundary conditions');
end


function inputs = build_default_inputs()

POUND_TO_NEWTON = 4.4482216152605;

inputs = struct();

inputs.section = struct();

inputs.section.h1 = 0.0300;
inputs.section.h2 = 0.0600;
inputs.section.t1 = 0.0020;
inputs.section.t2 = 0.0020;

inputs.beam = struct();

inputs.beam.length = 1.0000;
inputs.beam.elastic_modulus = 68.9e9;

inputs.loads = struct();

inputs.loads.selected_load_lb = 12.0;
inputs.loads.tip_force_y = 0.0;
inputs.loads.pound_to_newton = POUND_TO_NEWTON;

inputs.analysis = struct();

inputs.analysis.section_positions = (0.0000:0.0010:1.0000).';
inputs.analysis.sample_positions = [ ...
    0.0348; ...
    0.2761; ...
    0.5047; ...
    0.7333; ...
    0.9746];
inputs.analysis.stress_points = [ ...
    0.0000, 0.0150; ...
    0.0200, 0.0000];

inputs.experimental = struct();

inputs.experimental.load_levels_lb = [6.0, 12.0];
inputs.experimental.strain_micro = [ ...
    161.0, 309.0; ...
    68.0, 136.0; ...
    48.0, 99.0; ...
    31.0, 69.0; ...
```

```
    16.0, 32.0; ...
    -334.0, -630.0; ...
    -164.0, -326.0; ...
    -121.0, -239.0; ...
    -59.0, -119.0; ...
    -28.0, -58.0];
inputs.experimental.vertical_disp_mm = [ ...
    -0.619, -0.933; ...
    -6.578, -10.048; ...
    -12.127, -20.136; ...
    -18.576, -31.087; ...
    -25.513, -44.222];
inputs.experimental.horizontal_disp_mm = [ ...
    -0.181, -0.069; ...
    -0.638, -0.730; ...
    -2.070, -3.257; ...
    -2.757, -5.306; ...
    -4.555, -8.324];
end

% This function validates the required input fields.
function validate_inputs(inputs)

if ~isfield(inputs, 'section') ...
        || ~isfield(inputs, 'beam') ...
        || ~isfield(inputs, 'loads') ...
        || ~isfield(inputs, 'analysis') ...
        || ~isfield(inputs, 'experimental')
    error('The inputs structure is missing required fields.');
end

required_section = {'h1', 'h2', 't1', 't2'};

for field_ind = 1:numel(required_section)
    current_field = required_section{field_ind};

    if ~isfield(inputs.section, current_field)
        error('The section field %s is missing.', current_field);
    end
end

if inputs.section.h1 <= 0.0 || inputs.section.h2 <= 0.0
    error('The section heights must be positive.');
end

if inputs.section.t1 <= 0.0 || inputs.section.t2 <= 0.0
    error('The section thicknesses must be positive.');
end

if inputs.beam.length <= 0.0
    error('The beam length must be positive.');
end

if inputs.beam.elastic_modulus <= 0.0
```

```
    error('The elastic modulus must be positive.');
end

if any(inputs.analysis.section_positions < 0.0) ...
        || any(inputs.analysis.section_positions > inputs.beam.length)
    error('All section positions must lie between 0 and beam length.');
end

if any(inputs.analysis.sample_positions < 0.0) ...
        || any(inputs.analysis.sample_positions > inputs.beam.length)
    error('All sample positions must lie between 0 and beam length.');
end

if size(inputs.analysis.stress_points, 2) ~= 2
    error('The stress_points array must have two columns.');
end

if ~ismember(inputs.loads.selected_load_lb, ...
        inputs.experimental.load_levels_lb)
    error('The selected load must match an experimental load level.');
end
end

% This function returns the selected experimental load case.
function load_case = get_selected_load_case(loads, experimental)

column_ind = find( ...
    abs(experimental.load_levels_lb - loads.selected_load_lb) ...
    < 1.0e-12, ...
    1);

load_case = struct();

load_case.load_lb = loads.selected_load_lb;
load_case.tip_force_y = loads.tip_force_y;
load_case.tip_force_z = -loads.selected_load_lb ...
    * loads.pound_to_newton;
load_case.column_ind = column_ind;
end

% This function calculates the L-section properties.
function section = get_l_section_properties(section)

area_1 = section.t1 * section.h1;
area_2 = section.h2 * section.t2;
total_area = area_1 + area_2;

y_1 = section.t1 / 2.0;
z_1 = section.h1 / 2.0;
y_2 = section.t1 + section.h2 / 2.0;
z_2 = section.t2 / 2.0;

y_centroid = (area_1 * y_1 + area_2 * y_2) / total_area;
z_centroid = (area_1 * z_1 + area_2 * z_2) / total_area;
```

```
inertia_y_1_local = section.t1 * section.h1^3 / 12.0;
inertia_z_1_local = section.h1 * section.t1^3 / 12.0;
inertia_y_2_local = section.h2 * section.t2^3 / 12.0;
inertia_z_2_local = section.t2 * section.h2^3 / 12.0;

delta_y_1 = y_1 - y_centroid;
delta_z_1 = z_1 - z_centroid;
delta_y_2 = y_2 - y_centroid;
delta_z_2 = z_2 - z_centroid;

inertia_y = inertia_y_1_local ...
    + area_1 * delta_z_1^2 ...
    + inertia_y_2_local ...
    + area_2 * delta_z_2^2;

inertia_z = inertia_z_1_local ...
    + area_1 * delta_y_1^2 ...
    + inertia_z_2_local ...
    + area_2 * delta_y_2^2;

inertia_yz = area_1 * delta_y_1 * delta_z_1 ...
    + area_2 * delta_y_2 * delta_z_2;

denominator = inertia_y * inertia_z - inertia_yz^2;

if denominator <= 0.0
    error('The inertia denominator must be positive.');
end

section.area_1 = area_1;
section.area_2 = area_2;
section.area = total_area;
section.y_1 = y_1;
section.z_1 = z_1;
section.y_2 = y_2;
section.z_2 = z_2;
section.y_centroid = y_centroid;
section.z_centroid = z_centroid;
section.inertia_y = inertia_y;
section.inertia_z = inertia_z;
section.inertia_yz = inertia_yz;
section.denominator = denominator;
section.k_y = inertia_y / denominator;
section.k_z = inertia_z / denominator;
section.k_yz = inertia_yz / denominator;
end

% This function checks that the stress points lie in the section.
function validate_stress_points(stress_points, section)

num_points = size(stress_points, 1);

for point_ind = 1:num_points
```

```
    point_y = stress_points(point_ind, 1);
    point_z = stress_points(point_ind, 2);

    if ~is_in_l_section(point_y, point_z, section)
        error( ...
            'Stress point (%g, %g) is outside the L-section.', ...
            point_y, ...
            point_z);
    end
end
end

% This function checks whether a point lies inside the L-section.
function is_inside = is_in_l_section(point_y, point_z, section)

inside_vertical = point_y >= 0.0 ...
    && point_y <= section.t1 ...
    && point_z >= 0.0 ...
    && point_z <= section.h1;

inside_horizontal = point_y >= section.t1 ...
    && point_y <= section.t1 + section.h2 ...
    && point_z >= 0.0 ...
    && point_z <= section.t2;

is_inside = inside_vertical || inside_horizontal;
end

% This function builds one full theory case.
function case_results = get_case_results( ...
        section, ...
        beam, ...
        load_case, ...
        analysis, ...
        boundary_constants)

section_positions = analysis.section_positions(:);
sample_positions = analysis.sample_positions(:);

moment_y = get_bending_moment_y( ...
    beam.length, ...
    load_case.tip_force_z, ...
    section_positions);

moment_z = get_bending_moment_z( ...
    beam.length, ...
    load_case.tip_force_y, ...
    section_positions);

sample_moment_y = get_bending_moment_y( ...
    beam.length, ...
    load_case.tip_force_z, ...
    sample_positions);
```

```
sample_moment_z = get_bending_moment_z( ...
    beam.length, ...
    load_case.tip_force_y, ...
    sample_positions);

stress_values_pa = get_stress_history( ...
    section, ...
    moment_y, ...
    moment_z, ...
    analysis.stress_points);

sample_stress_pa = get_stress_history( ...
    section, ...
    sample_moment_y, ...
    sample_moment_z, ...
    analysis.stress_points);

[horizontal_disp_m, vertical_disp_m] = get_tip_load_deflections( ...
    section, ...
    beam, ...
    load_case, ...
    section_positions, ...
    boundary_constants);

[sample_horizontal_m, sample_vertical_m] = get_tip_load_deflections( ...
    section, ...
    beam, ...
    load_case, ...
    sample_positions, ...
    boundary_constants);

case_results = struct();

case_results.section_positions = section_positions;
case_results.boundary_constants = boundary_constants;

case_results.theory = struct();

case_results.theory.sample_positions = sample_positions;
case_results.theory.moment_y = moment_y;
case_results.theory.moment_z = moment_z;
case_results.theory.stress_side_pa = stress_values_pa(:, 1);
case_results.theory.stress_base_pa = stress_values_pa(:, 2);
case_results.theory.strain_side_micro = ...
    stress_values_pa(:, 1) / beam.elastic_modulus * 1.0e6;
case_results.theory.strain_base_micro = ...
    stress_values_pa(:, 2) / beam.elastic_modulus * 1.0e6;
case_results.theory.horizontal_disp_m = horizontal_disp_m;
case_results.theory.vertical_disp_m = vertical_disp_m;
case_results.theory.sample_stress_side_pa = sample_stress_pa(:, 1);
case_results.theory.sample_stress_base_pa = sample_stress_pa(:, 2);
case_results.theory.sample_strain_side_micro = ...
    sample_stress_pa(:, 1) / beam.elastic_modulus * 1.0e6;
case_results.theory.sample_strain_base_micro = ...
```

```
    sample_stress_pa(:, 2) / beam.elastic_modulus * 1.0e6;
case_results.theory.sample_horizontal_disp_m = sample_horizontal_m;
case_results.theory.sample_vertical_disp_m = sample_vertical_m;
case_results.theory.stress_all_pa = [ ...
    sample_stress_pa(:, 1); ...
    sample_stress_pa(:, 2)];
end

% This function calculates the y-axis bending moment.
function moment_y = get_bending_moment_y( ...
        beam_length, ...
        tip_force_z, ...
        positions)

moment_y = -tip_force_z .* (beam_length - positions);
end

% This function calculates the z-axis bending moment.
function moment_z = get_bending_moment_z( ...
        beam_length, ...
        tip_force_y, ...
        positions)

moment_z = tip_force_y .* (beam_length - positions);
end

% This function calculates the bending stress history in pascals.
function stress_values = get_stress_history( ...
        section, ...
        moment_y, ...
        moment_z, ...
        stress_points)

num_sections = numel(moment_y);
num_points = size(stress_points, 1);

stress_values = zeros(num_sections, num_points);

for point_ind = 1:num_points
    point_y = stress_points(point_ind, 1);
    point_z = stress_points(point_ind, 2);

    y_rel = point_y - section.y_centroid;
    z_rel = point_z - section.z_centroid;

    term_y = moment_z .* section.inertia_y ...
        - moment_y .* section.inertia_yz;

    term_z = moment_y .* section.inertia_z ...
        - moment_z .* section.inertia_yz;

    stress_values(:, point_ind) = ( ...
        term_y .* y_rel ...
        + term_z .* z_rel) ./ section.denominator;
```

```
end
end

% This function calculates the deflections for any boundary constants.
function [horizontal_disp, vertical_disp] = get_tip_load_deflections( ...
        section, ...
        beam, ...
        load_case, ...
        positions, ...
        boundary_constants)

shape_function = positions.^2 .* (3.0 * beam.length - positions) ...
    / 6.0;

horizontal_coef = ( ...
    section.inertia_y * load_case.tip_force_y ...
    - section.inertia_yz * load_case.tip_force_z) ...
    / (beam.elastic_modulus * section.denominator);

vertical_coef = ( ...
    section.inertia_z * load_case.tip_force_z ...
    - section.inertia_yz * load_case.tip_force_y) ...
    / (beam.elastic_modulus * section.denominator);

horizontal_disp = horizontal_coef .* shape_function ...
    + boundary_constants(3) .* positions ...
    + boundary_constants(4);

vertical_disp = vertical_coef .* shape_function ...
    + boundary_constants(1) .* positions ...
    + boundary_constants(2);
end

% This function returns the selected experimental measurements.
function experimental = get_experimental_results( ...
        load_case, ...
        analysis, ...
        elastic_modulus, ...
        experimental_inputs)

column_ind = load_case.column_ind;

strain_all_micro = experimental_inputs.strain_micro(:, column_ind);

strain_side_micro = strain_all_micro(1:5);
strain_base_micro = strain_all_micro(6:10);

stress_all_mpa = get_experimental_stress_mpa( ...
    strain_all_micro, ...
    elastic_modulus);

vertical_disp_mm = experimental_inputs.vertical_disp_mm(:, column_ind);
horizontal_disp_mm = experimental_inputs.horizontal_disp_mm(:, column_ind);
```

```
experimental = struct();

experimental.sample_positions = analysis.sample_positions(:);
experimental.strain_all_micro = strain_all_micro;
experimental.strain_side_micro = strain_side_micro;
experimental.strain_base_micro = strain_base_micro;
experimental.stress_all_mpa = stress_all_mpa;
experimental.stress_side_mpa = stress_all_mpa(1:5);
experimental.stress_base_mpa = stress_all_mpa(6:10);
experimental.vertical_disp_mm = vertical_disp_mm;
experimental.vertical_disp_m = vertical_disp_mm / 1.0e3;
experimental.horizontal_disp_mm = horizontal_disp_mm;
experimental.horizontal_disp_m = horizontal_disp_mm / 1.0e3;
end

% This function converts experimental strain to stress in megapascals.
function stress_mpa = get_experimental_stress_mpa( ...
        strain_micro, ...
        elastic_modulus)

stress_mpa = elastic_modulus .* strain_micro * 1.0e-12;
end

% This function calculates the required APRE values.
function apre_summary = get_apre_summary(case_results, experimental)

apre_summary = struct();

apre_summary.vertical_disp_percent = get_group_apre( ...
    experimental.vertical_disp_mm, ...
    case_results.theory.sample_vertical_disp_m * 1.0e3);

apre_summary.horizontal_disp_percent = get_group_apre( ...
    experimental.horizontal_disp_mm, ...
    case_results.theory.sample_horizontal_disp_m * 1.0e3);

apre_summary.stress_side_percent = get_group_apre( ...
    experimental.stress_side_mpa, ...
    case_results.theory.sample_stress_side_pa / 1.0e6);

apre_summary.stress_base_percent = get_group_apre( ...
    experimental.stress_base_mpa, ...
    case_results.theory.sample_stress_base_pa / 1.0e6);

apre_summary.strain_side_percent = get_group_apre( ...
    experimental.strain_side_micro, ...
    case_results.theory.sample_strain_side_micro);

apre_summary.strain_base_percent = get_group_apre( ...
    experimental.strain_base_micro, ...
    case_results.theory.sample_strain_base_micro);
end

% This function calculates RMSE values for all six data groups.
```

```
function rmse_summary = get_rmse_summary(case_results, experimental)

    rmse_summary = struct();

    rmse_summary.vertical_disp_mm = get_group_rmse( ...
        experimental.vertical_disp_mm, ...
        case_results.theory.sample_vertical_disp_m * 1.0e3);

    rmse_summary.horizontal_disp_mm = get_group_rmse( ...
        experimental.horizontal_disp_mm, ...
        case_results.theory.sample_horizontal_disp_m * 1.0e3);

    rmse_summary.stress_side_mpa = get_group_rmse( ...
        experimental.stress_side_mpa, ...
        case_results.theory.sample_stress_side_pa / 1.0e6);

    rmse_summary.stress_base_mpa = get_group_rmse( ...
        experimental.stress_base_mpa, ...
        case_results.theory.sample_stress_base_pa / 1.0e6);

    rmse_summary.strain_side_micro = get_group_rmse( ...
        experimental.strain_side_micro, ...
        case_results.theory.sample_strain_side_micro);

    rmse_summary.strain_base_micro = get_group_rmse( ...
        experimental.strain_base_micro, ...
        case_results.theory.sample_strain_base_micro);
end

% This function evaluates the APRE definition from the lab.
function apre_percent = get_group_apre(measured_values, calculated_values)

denominator = sum(calculated_values);

if abs(denominator) < 1.0e-12
    apre_percent = NaN;
else
    apre_percent = 100.0 * sum(measured_values - calculated_values) ...
        / denominator;
end
end

% This function evaluates the root-mean-square error.
function rmse_value = get_group_rmse(measured_values, calculated_values)

residual = measured_values - calculated_values;
rmse_value = sqrt(mean(residual.^2));
end

% This function fits Young''s modulus from the strain-gage data.
function best_modulus = fit_elastic_modulus( ...
        theory_stress_all_pa, ...
        experimental_strain_all_micro)
```

```
experimental_strain = experimental_strain_all_micro * 1.0e-6;

numerator = dot(experimental_strain, theory_stress_all_pa);
denominator = dot(experimental_strain, experimental_strain);

if denominator <= 0.0
    error('The strain fit denominator must be positive.');
end

best_modulus = numerator / denominator;
end

% This function fits the non-perfect clamping boundary constants.
function boundary_constants = fit_boundary_conditions( ...
        sample_positions, ...
        baseline_vertical_m, ...
        baseline_horizontal_m, ...
        experimental_vertical_m, ...
        experimental_horizontal_m)

vertical_system = [sample_positions, ones(size(sample_positions))];
vertical_rhs = experimental_vertical_m - baseline_vertical_m;
vertical_fit = vertical_system \ vertical_rhs;

horizontal_system = [sample_positions, ones(size(sample_positions))];
horizontal_rhs = experimental_horizontal_m - baseline_horizontal_m;
horizontal_fit = horizontal_system \ horizontal_rhs;

boundary_constants = [ ...
    vertical_fit(1); ...
    vertical_fit(2); ...
    horizontal_fit(1); ...
    horizontal_fit(2)];
end

% This function fits both Young''s modulus and boundary conditions.
function combined_fit = fit_combined_model( ...
        section, ...
        beam, ...
        load_case, ...
        analysis, ...
        experimental)

baseline_case = get_case_results( ...
    section, ...
    beam, ...
    load_case, ...
    analysis, ...
    zeros(4, 1));

initial_boundary = fit_boundary_conditions( ...
    baseline_case.theory.sample_positions, ...
    baseline_case.theory.sample_vertical_disp_m, ...
    baseline_case.theory.sample_horizontal_disp_m, ...
```

```
    experimental.vertical_disp_m, ...
    experimental.horizontal_disp_m);

initial_guess = [ ...
    log(beam.elastic_modulus); ...
    initial_boundary];

options = optimset('Display', 'off');

solution = fminsearch( ...
    @(variables) get_combined_fit_measure( ...
        variables, ...
        section, ...
        beam, ...
        load_case, ...
        analysis, ...
        experimental), ...
    initial_guess, ...
    options);

combined_fit = struct();

combined_fit.elastic_modulus = exp(solution(1));
combined_fit.boundary_constants = solution(2:5);
combined_fit.measure = get_combined_fit_measure( ...
    solution, ...
    section, ...
    beam, ...
    load_case, ...
    analysis, ...
    experimental);
end

% This function evaluates the extra-credit combined fit measure.
function measure = get_combined_fit_measure( ...
        variables, ...
        section, ...
        beam, ...
        load_case, ...
        analysis, ...
        experimental)

trial_beam = beam;

trial_beam.elastic_modulus = exp(variables(1));

trial_case = get_case_results( ...
    section, ...
    trial_beam, ...
    load_case, ...
    analysis, ...
    variables(2:5));

strain_residual = [ ...
```

```
    experimental.strain_side_micro ...
    - trial_case.theory.sample_strain_side_micro; ...
    experimental.strain_base_micro ...
    - trial_case.theory.sample_strain_base_micro];

vertical_residual = experimental.vertical_disp_mm ...
    - trial_case.theory.sample_vertical_disp_m * 1.0e3;

horizontal_residual = experimental.horizontal_disp_mm ...
    - trial_case.theory.sample_horizontal_disp_m * 1.0e3;

strain_scale = max(max(abs(experimental.strain_all_micro)), 1.0e-12);
vertical_scale = max(max(abs(experimental.vertical_disp_mm)), 1.0e-12);
horizontal_scale = max( ...
    max(abs(experimental.horizontal_disp_mm)), ...
    1.0e-12);

measure = mean((strain_residual / strain_scale).^2) ...
    + mean((vertical_residual / vertical_scale).^2) ...
    + mean((horizontal_residual / horizontal_scale).^2);
end

% This function builds a summary table of APRE values.
function apre_table = build_apre_table( ...
        baseline_apre, ...
        modulus_apre, ...
        boundary_apre, ...
        combined_apre)

apre_table = table( ...
    [baseline_apre.vertical_disp_percent; ...
    modulus_apre.vertical_disp_percent; ...
    boundary_apre.vertical_disp_percent; ...
    combined_apre.vertical_disp_percent], ...
    [baseline_apre.horizontal_disp_percent; ...
    modulus_apre.horizontal_disp_percent; ...
    boundary_apre.horizontal_disp_percent; ...
    combined_apre.horizontal_disp_percent], ...
    [baseline_apre.stress_side_percent; ...
    modulus_apre.stress_side_percent; ...
    boundary_apre.stress_side_percent; ...
    combined_apre.stress_side_percent], ...
    [baseline_apre.stress_base_percent; ...
    modulus_apre.stress_base_percent; ...
    boundary_apre.stress_base_percent; ...
    combined_apre.stress_base_percent], ...
    [baseline_apre.strain_side_percent; ...
    modulus_apre.strain_side_percent; ...
    boundary_apre.strain_side_percent; ...
    combined_apre.strain_side_percent], ...
    [baseline_apre.strain_base_percent; ...
    modulus_apre.strain_base_percent; ...
    boundary_apre.strain_base_percent; ...
    combined_apre.strain_base_percent], ...
```

```
    'VariableNames', { ...
    'vertical_disp_percent', ...
    'horizontal_disp_percent', ...
    'stress_side_percent', ...
    'stress_base_percent', ...
    'strain_side_percent', ...
    'strain_base_percent'}, ...
    'RowNames', { ...
    'baseline', ...
    'fitted_modulus', ...
    'fitted_boundary', ...
    'combined_fit'});
end

% This function builds a stress and strain comparison table.
function stress_table = build_stress_comparison_table( ...
        baseline_case, ...
        baseline_experimental, ...
        modulus_case, ...
        modulus_experimental)

gage_number = (1:10).';

x_value = [ ...
    baseline_case.theory.sample_positions; ...
    baseline_case.theory.sample_positions];

y_value = [ ...
    zeros(5, 1); ...
    0.0200 * ones(5, 1)];

z_value = [ ...
    0.0150 * ones(5, 1); ...
    zeros(5, 1)];

baseline_stress_mpa = [ ...
    baseline_case.theory.sample_stress_side_pa / 1.0e6; ...
    baseline_case.theory.sample_stress_base_pa / 1.0e6];

baseline_strain_micro = [ ...
    baseline_case.theory.sample_strain_side_micro; ...
    baseline_case.theory.sample_strain_base_micro];

modulus_stress_mpa = [ ...
    modulus_case.theory.sample_stress_side_pa / 1.0e6; ...
    modulus_case.theory.sample_stress_base_pa / 1.0e6];

modulus_strain_micro = [ ...
    modulus_case.theory.sample_strain_side_micro; ...
    modulus_case.theory.sample_strain_base_micro];

stress_table = table( ...
    gage_number, ...
    x_value, ...
```

```
    y_value, ...
    z_value, ...
    baseline_experimental.strain_all_micro, ...
    baseline_experimental.stress_all_mpa, ...
    baseline_stress_mpa, ...
    baseline_strain_micro, ...
    modulus_experimental.stress_all_mpa, ...
    modulus_stress_mpa, ...
    modulus_strain_micro, ...
    'VariableNames', { ...
    'gage_number', ...
    'x_value', ...
    'y_value', ...
    'z_value', ...
    'exp_strain_micro', ...
    'exp_stress_nominal_mpa', ...
    'theory_stress_nominal_mpa', ...
    'theory_strain_nominal_micro', ...
    'exp_stress_fitted_mpa', ...
    'theory_stress_fitted_mpa', ...
    'theory_strain_fitted_micro'});
end

% This function builds a displacement comparison table.
function deflection_table = build_deflection_comparison_table( ...
        baseline_case, ...
        baseline_experimental, ...
        boundary_case, ...
        combined_case)

x_value = baseline_case.theory.sample_positions;

deflection_table = table( ...
    x_value, ...
    baseline_experimental.vertical_disp_mm, ...
    baseline_case.theory.sample_vertical_disp_m * 1.0e3, ...
    boundary_case.theory.sample_vertical_disp_m * 1.0e3, ...
    combined_case.theory.sample_vertical_disp_m * 1.0e3, ...
    baseline_experimental.horizontal_disp_mm, ...
    baseline_case.theory.sample_horizontal_disp_m * 1.0e3, ...
    boundary_case.theory.sample_horizontal_disp_m * 1.0e3, ...
    combined_case.theory.sample_horizontal_disp_m * 1.0e3, ...
    'VariableNames', { ...
    'x_value', ...
    'exp_vertical_mm', ...
    'baseline_vertical_mm', ...
    'boundary_vertical_mm', ...
    'combined_vertical_mm', ...
    'exp_horizontal_mm', ...
    'baseline_horizontal_mm', ...
    'boundary_horizontal_mm', ...
    'combined_horizontal_mm'});
end
```

```
% This function prints the main Part 2 results.
function display_summary(results)

fprintf('\n');
fprintf('================ PROJECT PART 2 SUMMARY ================\n');
fprintf('Selected load = %.1f lb\n', results.load_case.load_lb);
fprintf('Nominal modulus = %.6e Pa\n', ...
    results.inputs.beam.elastic_modulus);
fprintf('Fitted modulus = %.6e Pa\n', results.best_modulus_pa);
fprintf('Combined-fit modulus = %.6e Pa\n', ...
    results.combined_fit.elastic_modulus);
fprintf('\n');
fprintf('============= FITTED BOUNDARY CONDITIONS ================\n');
fprintf('dw_dx(0) = %.6e rad\n', results.boundary_constants(1));
fprintf('w(0) = %.6e m\n', results.boundary_constants(2));
fprintf('dv_dx(0) = %.6e rad\n', results.boundary_constants(3));
fprintf('v(0) = %.6e m\n', results.boundary_constants(4));
fprintf('\n');
fprintf('=========== COMBINED-FIT BOUNDARY CONDITIONS ============\n');
fprintf('dw_dx(0) = %.6e rad\n', ...
    results.combined_fit.boundary_constants(1));
fprintf('w(0) = %.6e m\n', ...
    results.combined_fit.boundary_constants(2));
fprintf('dv_dx(0) = %.6e rad\n', ...
    results.combined_fit.boundary_constants(3));
fprintf('v(0) = %.6e m\n', ...
    results.combined_fit.boundary_constants(4));
fprintf('Combined measure = %.6e\n', results.combined_fit.measure);
fprintf('\n');
fprintf('================== APRE SUMMARY [%%] ==================\n');
disp(results.apre_table);
fprintf('\n');
disp('================ RMSE SUMMARY ====================');
disp(results.rmse_table);
end

% This function creates one six-panel comparison figure.
function plot_comparison_set(case_results, experimental, figure_title_text)

sample_positions = case_results.theory.sample_positions;

figure('Name', figure_title_text);

tiledlayout(3, 2);

nexttile;

plot( ...
    case_results.section_positions, ...
    case_results.theory.vertical_disp_m * 1.0e3, ...
    'LineWidth', 1.5);

hold on;
```

```
plot( ...
    sample_positions, ...
    experimental.vertical_disp_mm, ...
    'o', ...
    'LineWidth', 1.5, ...
    'MarkerSize', 6);

grid on;
xlabel('x (m)');
ylabel('w (mm)');
title('Vertical displacement');
legend('Theory', 'Experiment', 'Location', 'best');

nexttile;

plot( ...
    case_results.section_positions, ...
    case_results.theory.horizontal_disp_m * 1.0e3, ...
    'LineWidth', 1.5);

hold on;

plot( ...
    sample_positions, ...
    experimental.horizontal_disp_mm, ...
    'o', ...
    'LineWidth', 1.5, ...
    'MarkerSize', 6);

grid on;
xlabel('x (m)');
ylabel('v (mm)');
title('Horizontal displacement');
legend('Theory', 'Experiment', 'Location', 'best');

nexttile;

plot( ...
    case_results.section_positions, ...
    case_results.theory.stress_side_pa / 1.0e6, ...
    'LineWidth', 1.5);

hold on;

plot( ...
    sample_positions, ...
    experimental.stress_side_mpa, ...
    'o', ...
    'LineWidth', 1.5, ...
    'MarkerSize', 6);

grid on;
xlabel('x (m)');
ylabel('\sigma_{xx} (MPa)');
```

```
title('Stress at side gages');
legend('Theory', 'Experiment', 'Location', 'best');

nexttile;

plot( ...
    case_results.section_positions, ...
    case_results.theory.stress_base_pa / 1.0e6, ...
    'LineWidth', 1.5);

hold on;

plot( ...
    sample_positions, ...
    experimental.stress_base_mpa, ...
    'o', ...
    'LineWidth', 1.5, ...
    'MarkerSize', 6);

grid on;
xlabel('x (m)');
ylabel('\sigma_{xx} (MPa)');
title('Stress at base gages');
legend('Theory', 'Experiment', 'Location', 'best');

nexttile;

plot( ...
    case_results.section_positions, ...
    case_results.theory.strain_side_micro, ...
    'LineWidth', 1.5);

hold on;

plot( ...
    sample_positions, ...
    experimental.strain_side_micro, ...
    'o', ...
    'LineWidth', 1.5, ...
    'MarkerSize', 6);

grid on;
xlabel('x (m)');
ylabel('\epsilon (\mu\epsilon)');
title('Strain at side gages');
legend('Theory', 'Experiment', 'Location', 'best');

nexttile;

plot( ...
    case_results.section_positions, ...
    case_results.theory.strain_base_micro, ...
    'LineWidth', 1.5);
```

```
hold on;

plot( ...
    sample_positions, ...
    experimental.strain_base_micro, ...
    'o', ...
    'LineWidth', 1.5, ...
    'MarkerSize', 6);

grid on;
xlabel('x (m)');
ylabel('\epsilon (\mu\epsilon)');
title('Strain at base gages');
legend('Theory', 'Experiment', 'Location', 'best');

sgtitle(figure_title_text);
end
```

```
================= PROJECT PART 2 SUMMARY =================
Selected load = 12.0 lb
Nominal modulus = 6.890000e+10 Pa
Fitted modulus = 7.262615e+10 Pa
Combined-fit modulus = 7.803362e+10 Pa

============= FITTED BOUNDARY CONDITIONS =================
dw_dx(0) = -1.389185e-02 rad
w(0) = -1.909706e-03 m
dv_dx(0) = -1.519913e-03 rad
v(0) = 1.125730e-04 m

=========== COMBINED-FIT BOUNDARY CONDITIONS ============
dw_dx(0) = -1.765888e-02 rad
w(0) = -1.455116e-03 m
dv_dx(0) = -2.398532e-03 rad
v(0) = 2.188236e-04 m
Combined measure = 1.316184e-02

=================== APRE SUMMARY [%] ====================
                       vertical_disp_percent    horizontal_disp_percent
stress_side_percent    stress_base_percent    strain_side_percent
strain_base_percent
                      ______________________    ______________________
_____________________    ____________________    ____________________
_____________________

    baseline                    72.151
22.706                -34.578                -1.5952
-34.578               -1.5952
    fitted_modulus              81.461
29.342                 -31.04                 3.7266
-31.04                3.7266
    fitted_boundary         -1.4605e-15
3.7664e-15                -34.578                -1.5952
```

```
-34.578                -1.5952
    combined_fit              0.0027131
0.0062284                -25.906                  11.45
-25.906                  11.45


================ RMSE SUMMARY ====================
                       vertical_disp_mm    horizontal_disp_mm
stress_side_mpa    stress_base_mpa    strain_side_micro    strain_base_micro
                                  _________________    _________________
_______________    _______________    _________________    _________________

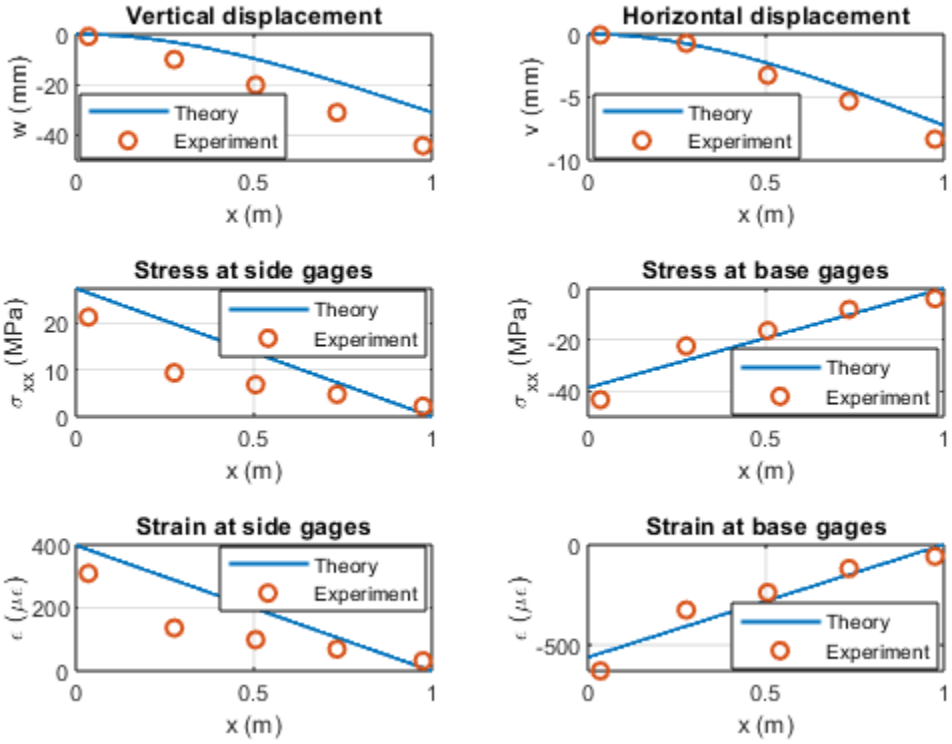
    baseline                10.097               0.8518
6.1876             4.2044               89.806               61.021
    fitted_modulus          10.897               1.0425
5.7396             4.5887                79.03               63.182
    fitted_boundary         1.1339              0.21173
6.1876             4.2044               89.806               61.021
    combined_fit           0.88417              0.22563
5.1448             5.6274                65.93               72.115


ans =

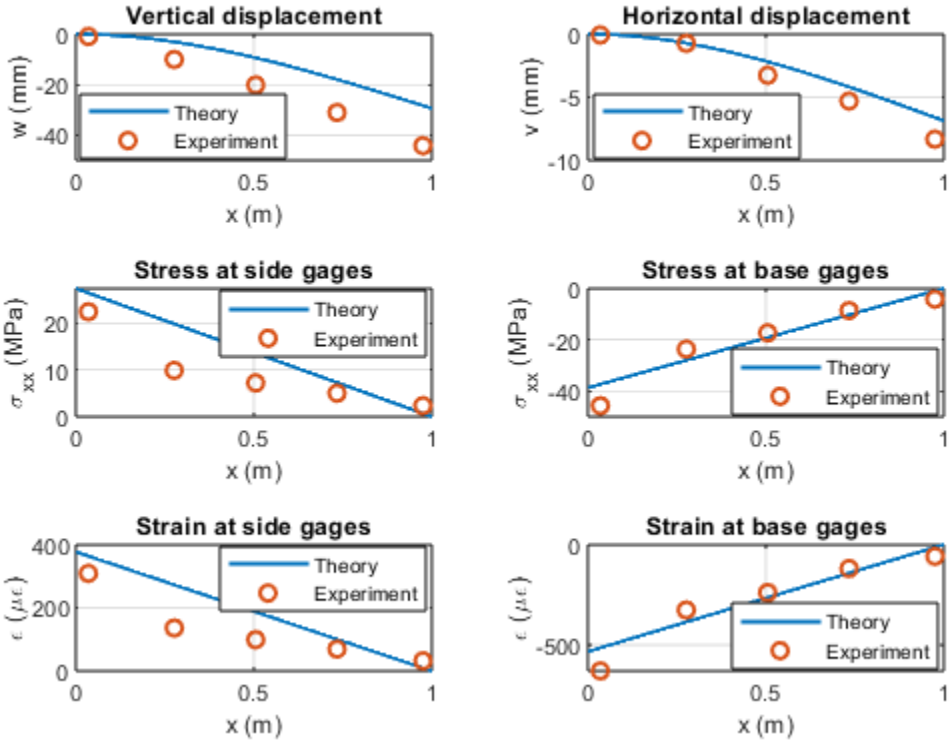
  struct with fields:

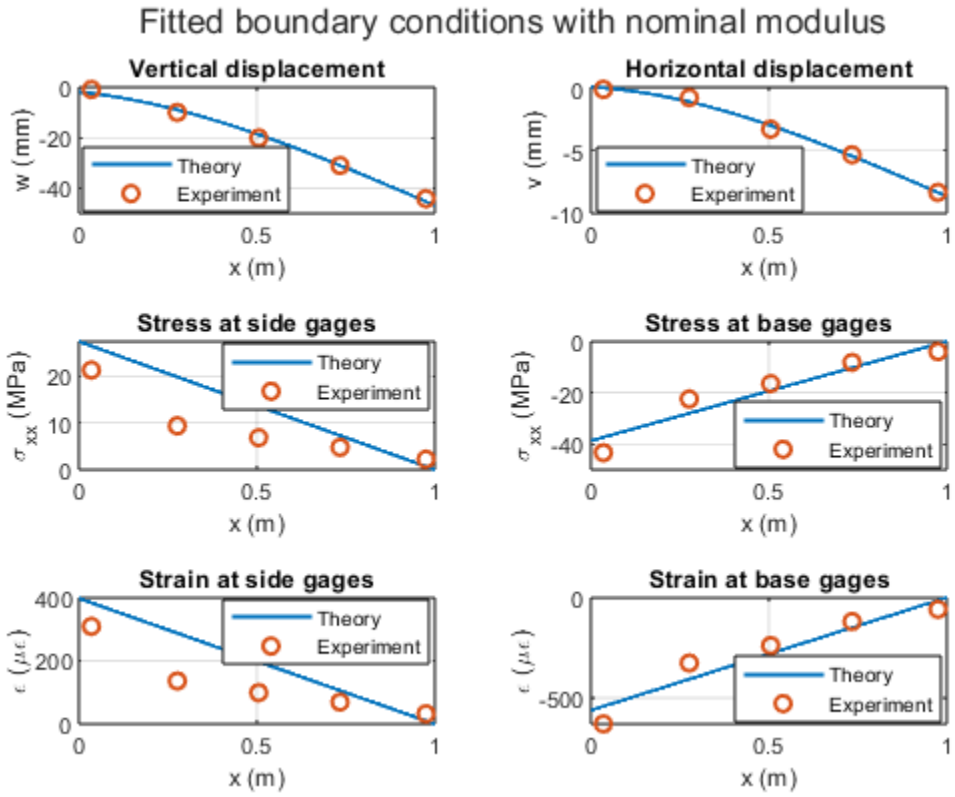
                  inputs: [1×1 struct]
                 section: [1×1 struct]
               load_case: [1×1 struct]
                baseline: [1×1 struct]
    baseline_experimental: [1×1 struct]
           baseline_apre: [1×1 struct]
         best_modulus_pa: 7.2626e+10
            modulus_case: [1×1 struct]
     modulus_experimental: [1×1 struct]
            modulus_apre: [1×1 struct]
       boundary_constants: [4×1 double]
           boundary_case: [1×1 struct]
           boundary_apre: [1×1 struct]
            combined_fit: [1×1 struct]
           combined_case: [1×1 struct]
    combined_experimental: [1×1 struct]
           combined_apre: [1×1 struct]
           baseline_rmse: [1×1 struct]
            modulus_rmse: [1×1 struct]
           boundary_rmse: [1×1 struct]
           combined_rmse: [1×1 struct]
              apre_table: [4×6 table]
              rmse_table: [4×6 table]
             stress_table: [10×11 table]
        deflection_table: [5×9 table]
```

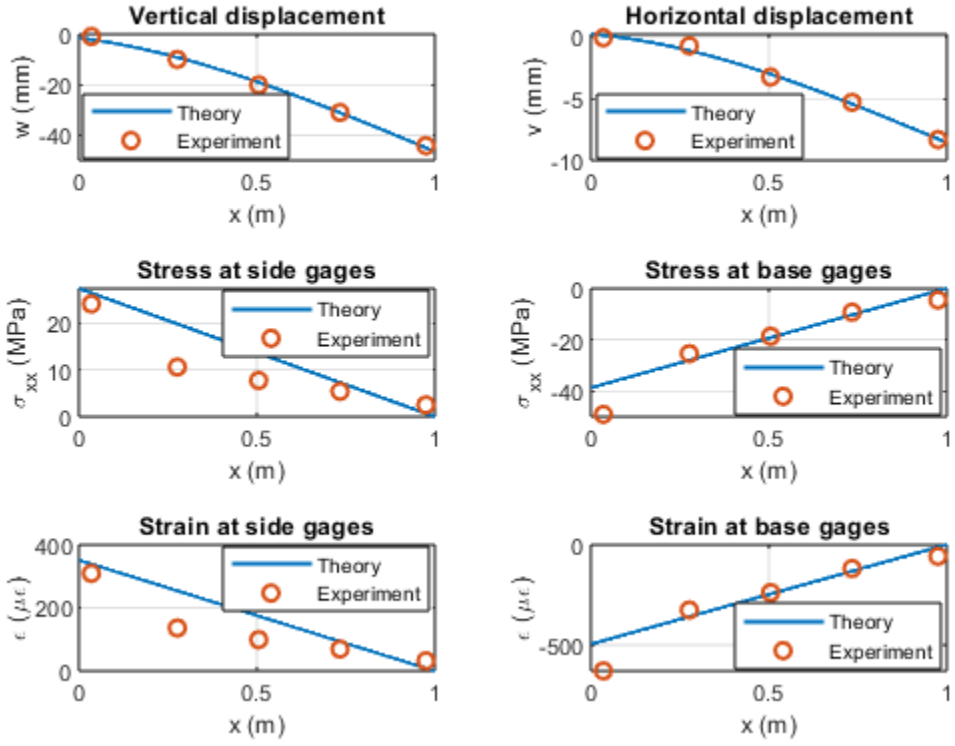
Perfect clamp with nominal modulus



Perfect clamp with fitted modulus




Fitted boundary conditions with nominal modulus
Vertical displacement
w (mm)
x (m)
Theory
Experiment
Horizontal displacement
v (mm)
Stress at side gages
σxx (MPa)
Stress at base gages
Strain at side gages
ε (με)
Strain at base gages


Combined fitted modulus and boundary conditions
Vertical displacement
w (mm)
x (m)
Theory
Experiment
Horizontal displacement
v (mm)
Stress at side gages
σxx (MPa)
Stress at base gages
Strain at side gages
ε (με)
Strain at base gages

AI Use Statement

For this assignment's preparation, the author utilized ChatGPT, a language model created by OpenAI. Within this assignment, ChatGPT was used for brainstorming, checking MATLAB code organization, identifying possible errors, polishing code, and suggesting debugging fixes.

Published with MATLAB® R2023b